МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет по лабораторной работе №1
по дисциплине «Искусственный интеллект и машинное обучение»

Выполнил студент Ромащенко Данил группы ИВТ-б-о-24-1

Подпись студента_____
Работа защищена « »_____20__г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Тема: «Работа с Jupyter Notebook, JupyterLab и Google Colab».

Цель работы: исследовать базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.

Ход работы

Вариант 17

Ссылка на репозиторий: https://github.com/l1N322/lab_1.git

1) Был создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирование Python.

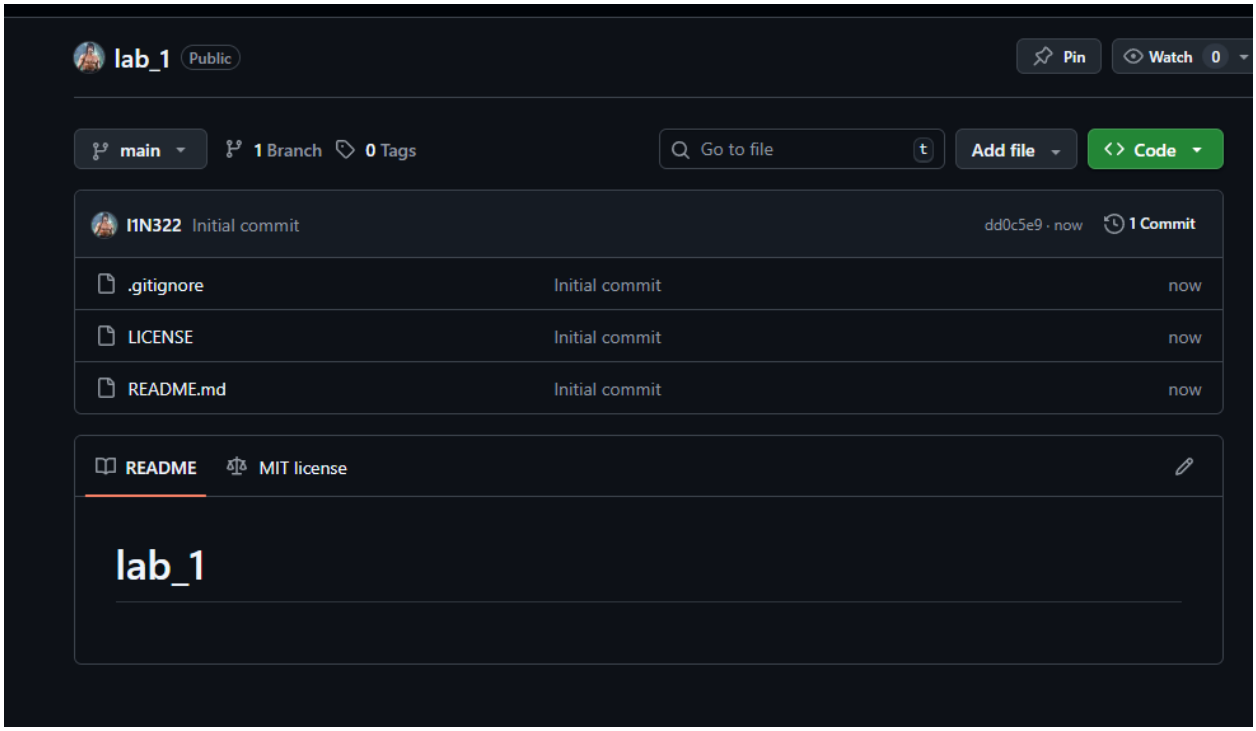

Рисунок 1. Созданный репозиторий

2) Был дополнен файл .gitignore необходимыми правилами.

```
# Covers JetBrains IDEs: IntelliJ, GoLand, RubyMine, PhpStorm, AppCode, PyCharm, CLion, Android Studio, WebStorm and Rider
# Reference: https://intellij-support.jetbrains.com/hc/en-us/articles/206544839

# User-specific stuff
.idea/**/workspace.xml
.idea/**/tasks.xml
.idea/**/usage.statistics.xml
.idea/**/dictionaries
.idea/**/shelf

# AWS User-specific
.idea/**/aws.xml

# Generated files
.idea/**/contentModel.xml

# Sensitive or high-churn files
.idea/**/dataSources/
.idea/**/dataSources.ids
.idea/**/dataSources.local.xml
.idea/**/sqlDataSources.xml
.idea/**/dynamic.xml
.idea/**/uiDesigner.xml
.idea/**/dbnavigator.xml

# Gradle
.idea/**/gradle.xml
.idea/**/libraries

# Gradle and Maven with auto-import
# When using Gradle or Maven with auto-import, you should exclude module files,
# since they will be recreated, and may cause churn.  Uncomment if using
# auto-import.
# .idea/artifacts
# .idea/compiler.xml
# .idea/jarRepositories.xml
```

Рисунок 2. Добавленные правила в файл .gitignore

3) Было выполнено клонирование созданного репозитория на компьютер.

```
MINGW64:/c/Users/USER

USER@topPC MINGW64 ~
$ git clone https://github.com/l1N322/lab_1.git
Cloning into 'lab_1'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), done.

USER@topPC MINGW64 ~
$ |
```

Рисунок 3. Клонирование репозитория

4) Были выполнены задания по своему варианту (вариант 17).

Создайте новую **Markdown-ячейку** и:

- Напишите заголовок `"Практическое задание №1"`.
- Добавьте **жирный** и *курсивный* текст.
- Создайте нумерованный и маркированный списки.
- Вставьте согласно индивидуального задания, например:
- Вставьте изображение через `![Описание](URL)`.

Создайте ячейку **Python-кода** и:

- Запросите у пользователя его имя с помощью `input()`.
- Выведите приветствие: `"Привет, <имя>! Добро пожаловать в JupyterLab / Google Colab!"`.
- Запустите ячейку (`Shift + Enter`).

Рисунок 4. Задание 1

Практическое задание №1

Жирный текст, *курсив*

Нумерованный список

1. Первый пунтк
2. Второй пунтк
3. Третий пунтк

Маркированный список ¶

- Первый
- Второй
- Третий
  - Подэлемент

Формула для вычисления собственных значений матрицы:

$|A - \lambda E|$, где E - единичная матрица того же размера, что A

Изображение:

Рисунок 5. Решение задания 1

1. Создайте и сохраните текстовый файл с помощью `open()`.
2. Запишите в него несколько строк текста.
3. Закройте файл и затем откройте его снова, считав содержимое и выведя на экран.
4. Проверьте, существует ли файл, используя `os.path.exists()`.
5. Удалите файл с помощью модуля `os`.

Рисунок 6. Задание 2

```
[3]: import os

     with open("file.txt", "w") as f:
         f.write("Первая строка\n")
         f.write("Вторая строка\n")

[4]: with open ("file.txt", "r") as f:
         content = f.read()
         print("Содержимое файла:\n", content)

     print("Файл существует:", os.path.exists("file.txt"))

     Содержимое файла:
      Первая строка
     Вторая строка

     Файл существует: True

[5]: os.remove("file.txt")
     print("Файл удалён")

     Файл удалён
```

Рисунок 7. Решение задания 2

1. Выведите список всех доступных магических команд (`%lsmagic`).
2. Используйте `%time` и `%%timeit` для измерения времени выполнения кода.
3. Создайте Python-скрипт в Jupyter (`%%writefile script.py`) и выполните его через `!python script.py`.
4. Выведите список файлов в текущей директории с помощью `%ls`.
5. Используйте `%history` для просмотра истории команд.

Рисунок 8. Задание 3

```
[17]: %%time
      %lsmagic

CPU times: total: 0 ns
Wall time: 37.7 µs

[17]: root
      line
      cell

[12]: %%writefile test_script.py
      for i in range(3):
          print(f"Итерация {i}")

Overwriting test_script.py

[15]: !python test_script.py

Итерация 0
Итерация 1
Итерация 2

[23]: %timeit x = [(i**10) for i in range(10)]

782 ns ± 5.7 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)

[24]: %ls

 Том в устройстве C имеет метку SSD_Kingston
 Серийный номер тома: 16FF-1A67

 Содержимое папки C:\Users\USER\notebooks

17.03.2026  23:19    <DIR>          .
17.03.2026  23:19    <DIR>          ..
17.03.2026  23:05    <DIR>          .ipynb_checkpoints
17.03.2026  22:32           100 757 1.ipynb
17.03.2026  23:04             1 838 2.ipynb
17.03.2026  23:19             7 412 3.ipynb
17.03.2026  23:16                56 test_script.py
               4 файлов        110 063 байт
               3 папок  69 956 407 296 байт свободно

[25]: %history

%lsmagic
%%writelife test_script.py
for i in range(3):
    print(f"Итерация {i}")
%%writefile test_script.py
for i in range(3):
    print(f"Итерация {i}")
!python test_script.py
%time sum(range(1000000))
```

Рисунок 9. Решение задания 3

1. Выведите список файлов в текущей директории с помощью `!ls`.
2. Проверьте, какой Python используется (`!which python`).
3. Создайте папку `test_folder` (`!mkdir test_folder`) и убедитесь, что она появилась.
4. Переместите файл в новую папку и удалите его.
5. Очистите вывод в ячейке (`!clear`).

Рисунок 10. Задание 4

```
!ls
sample_data

!which python
/usr/local/bin/python

!mkdir test_folder

!ls
sample_data  test_folder

with open("example.txt", "w") as f:
  f.write("Text\n")

!ls
example.txt  sample_data  test_folder

!mv example.txt test_folder/

!ls test_folder/
example.txt

!rm test_folder/example.txt

!ls test_folder/

!clear
```

Рисунок 11. Решение задания 4

1. **Подключите Google Drive** к Colab с помощью команды:

```python
from google.colab import drive
drive.mount('/content/drive')
```

Проверьте, что ваш диск успешно подключился, используя `!ls /content/drive/MyDrive`.

2. **Создайте и сохраните текстовый файл** в Google Drive:
   - Откройте файл `my_text_file.txt` в режиме записи.
   - Запишите в него несколько строк текста.
   - Закройте файл и убедитесь, что он появился в папке `MyDrive`.
3. **Прочитайте файл из Google Drive**:
   - Откройте ранее созданный файл.
   - Считайте его содержимое и выведите в ячейке.

---

4. **Создайте и сохраните CSV-файл вручную, используя Microsoft Excell или Libre Office Calc**:
   - Создайте список, в котором будут строки с данными о студентах (например, ФИО, возраст, группа).
   - Запишите этот список в CSV-файл в Google Drive вручную, используя стандартные методы записи в файл.

Рисунок 12. Задание 5

```python
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```python
!ls /content/drive/MyDrive
```

```
'Colab Notebooks'
```

```python
file_path = "/content/drive/MyDrive/my_text_file.txt"
with open(file_path, "w") as f:
    f.write("Это тестовый файл, сохраненый в Google Drive.\n")
    f.write("Вторая строка текста.")

print("Файл успешно сохранен в Google Drive.")
```

```
Файл успешно сохранен в Google Drive.
```

```python
with open (file_path, "r") as f:
    content = f.read()
    print("Содержимое файла:\n", content)
```

```
Содержимое файла:
 Текстовый файл
Он сохранён в Google Drive
```

```python
students = [
    ["Ромащенок Д.П.", 20, "ИВТ-б-о-24-1"],
    ["Аверин М.С.", 19, "ИВТ-б-о-24-1"],
    ["Грабарь А.П.", 19, "ИВТ-б-о-24-1"]
]

csv_path = "/content/drive/MyDrive/students.csv"
with open(csv_path, "w") as f:
    for student in students:
        f.write(",".join(map(str, student)) + "\n")
```

Рисунок 13. Решение задания 5

⏩ **Дополнительно:** попробуйте загрузить файл из Google Drive в Colab с помощью `files.upload()`, обработать его и сохранить обратно.

Рисунок 14. Дополнительное задание

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

from google.colab import files
uploaded = files.upload()

Выбрать файлы file.txt
file.txt(text/plain) - 34 bytes, last modified: 17.03.2026 - 100% done
Saving file.txt to file.txt

for filename in uploaded.keys():
    print(f"Загружен файл: {filename}")

with open(filename, "r") as f:
        content = f.read()
        print("Содержимое файла:\n", content)

Загружен файл: file.txt
Содержимое файла:
 Файл для обработки

processed_content = content + "\n"
processed_content += "Файл обработан в Google Colab.\n"
print(processed_content)

Файл для обработки
Файл обработан в Google Colab.

drive_path = "/content/drive/MyDrive/file.txt"

with open(drive_path, "w") as f:
    f.write(processed_content)

print(f"Обработанный файл сохранен в Google Drive:")
```

Рисунок 15. Решение дополнительного задания

Ответы на контрольные вопросы:

1) JupyterLab — это следующее поколение пользовательского интерфейса Project Jupyter. Его главное отличие — интегрированная среда разработки (IDE).

В отличие от классического Notebook с одним документом, JupyterLab позволяет открывать несколько ноутбуков, текстовых редакторов, терминалов и консолей в одном окне с вкладками и гибкой компоновкой.

JupyterLab имеет встроенный мощный файловый браузер для навигации, загрузки и управления файлами прямо в интерфейсе.

2) Чтобы создать новую рабочую среду (ноутбук) в JupyterLab нужно:

Нажать кнопку "+" на левой панели инструментов или в меню File -> New Launcher .

В открывшейся вкладке Launcher вы увидите раздел "Notebook".

Выберите ядро (kernels) для вашего ноутбука (например, Python 3).

3) Основные типы ячеек:

– Code (Код): Для написания и выполнения кода.

– Markdown: Для форматированного текста, заголовков, списков, формул LaTeX.

– Raw (Сырые): Текст, который выводится без преобразований.

Переключение: Выделите ячейку и используйте выпадающее меню на панели инструментов или горячие клавиши в командном режиме (синяя рамка): y для Code, m для Markdown, r для Raw.

4) Выполнить код можно кнопкой "Run" на панели инструментов или горячими клавишами:

– Shift + Enter: Выполнить ячейку и перейти к следующей.

– Ctrl + Enter: Выполнить ячейку и остаться на ней.

– Alt + Enter: Выполнить ячейку и вставить новую снизу.

5) Чтобы запустить терминал или текстовый редактор внутри JupyterLab нужно:

В разделе "Other" Launcher нажмите иконку "Terminal".

Текстовый редактор: В разделе "Other" выберите "Text File" для создания нового файла, или просто дважды кликните на существующий текстовый файл (.py, .md, .txt и др.) в файловом браузере .

6) Основной инструмент — файловый браузер (File Browser) на левой боковой панели. Он позволяет:

– Создавать, удалять, переименовывать, копировать папки и файлы.

– Загружать файлы с компьютера кнопкой "Upload Files" (иконка со стрелкой вверх) или перетаскиванием.

– Скачивать файлы на компьютер (правой кнопкой мыши -> "Download").

– Просматривать структуру проекта.

7) Управление ядром текущего ноутбука осуществляется через меню Kernel на верхней панели. Там доступны опции:

– Interrupt: Прервать выполнение кода.

– Restart: Перезапустить ядро (очистить все переменные).

– Restart & Run All: Перезапустить ядро и выполнить все ячейки.

– Change Kernel...: Сменить ядро на другое (например, с Python 3.8 на Python 3.10 или другое окружение).

8) Основные возможности системы вкладок и окон в интерфейсе JupyterLab:

Вкладки: Каждый открытый документ (ноутбук, терминал, редактор) отображается как вкладка.

Разделение (Split): Вы можете перетаскивать вкладки вправо, влево, вверх или вниз от центра, чтобы создавать новые панели и работать с несколькими документами бок о бок.

Изменение размеров: Границы между панелями можно перетаскивать для изменения размеров окон.

9) В JupyterLab (с ядром IPython) используются "магические" команды:

– %time: Измеряет время выполнения одной инструкции.

– %timeit: Запускает код несколько раз и выдает среднее время, лучше подходит для точных замеров небольших фрагментов.

– %%time: Измеряет время выполнения всей ячейки (многострочный код).

– %%timeit: Запускает всю ячейку несколько раз и усредняет результат.

10) С помощью магии ячеек (Cell Magics) можно запускать код на других языках, если соответствующие ядра установлены:

– %%bash (или %%script bash): Выполнить ячейку как скрипт Bash.

– %%html: Интерпретирует содержимое как HTML.

– %%javascript (или %%js): Выполнить код JavaScript.

– %%ruby: Выполнить код Ruby.

– %%perl: Выполнить код Perl.

11) Colab — это облачный сервис Google, не требующий установки и предоставляющий вычислительные ресурсы (включая GPU/TPU) бесплатно. JupyterLab обычно устанавливается и запускается локально или на сервере, который вы контролируете.

Colab тесно интегрирован с Google Диском. JupyterLab работает с файловой системой сервера, на котором запущен.

Colab имеет встроенные функции для совместного редактирования, как в Google Документах.

12) Чтобы создать новый ноутбук в Google Colab нужно:

– Перейти на сайт colab.research.google.com.

– В открывшемся окне нажмите кнопку "Новый ноутбук" (New Notebook) в нижней части диалога.

– Или через меню: Файл -> Создать -> Новый блокнот.

13) Типы ячеек в Google Colab такие же, как в Jupyter.

Переключение: Используйте выпадающее меню в правом верхнем углу ячейки (по умолчанию "Код" или "Текст").

14) Горячие клавиши полностью аналогичны Jupyter:

– Shift + Enter: Выполнить ячейку и перейти к следующей.

– Ctrl + Enter: Выполнить ячейку и остаться на ней.

– Alt + Enter: Выполнить ячейку и вставить новую снизу.

15) Способы загрузки и сохранения файлов в Google Colab:

– Загрузка с компьютера: Через файловый менеджер (панель слева -> иконка папки -> иконка "Загрузить").

– Сохранение на компьютер: Через файловый менеджер (правая кнопка мыши на файле -> Скачать).

– Монтирование Google Диска: Позволяет читать и сохранять файлы прямо на вашем Диске.

– Загрузка/сохранение по URL: С помощью команд !wget <URL> или !curl -O <URL> для загрузки, и интеграцией с google.colab для сохранения.

16) Чтобы подключить Google Drive к Google Colab и работать с файлами нужно выполнить в ячейке код:

```
python
from google.colab import drive
drive.mount('/content/drive')
```

После выполнения вы перейти по ссылке, выбрать аккаунт Google и скопируете код авторизации в поле ввода. После этого ваш Диск будет доступен по пути /content/drive/MyDrive/.

17) Самый простой способ — использовать встроенные функции вкладки "Файлы":

Откройте панель "Файлы" (иконка папки слева).

Нажмите на иконку "Загрузить в хранилище сеанса" (первая иконка с изображением листа бумаги и стрелки вверх).

Выберите файл на своем компьютере.

Альтернативный способ — код с использованием библиотеки google.colab:

python

```
from google.colab import files
uploaded = files.upload()
```

Загруженный файл будет в словаре uploaded

18) Чтобы посмотреть список файлов, хранящихся в среде Google Colab нужно:

Использовать левую боковую панель "Файлы" (иконка папки). Там отображается файловая система виртуальной машины.

Или выполнить магическую команду оболочки в ячейке: !ls (для текущей директории, обычно /content/). Чтобы посмотреть все с подробностями: !ls -la.

19) Colab также работает на базе IPython, поэтому все магические команды из JupyterLab работают точно так же:

- %time
- %timeit
- %%time
- %%time

20) Чтобы изменить тип аппаратного ускорителя:

Откройте меню "Среда выполнения" (Runtime).

Выберите пункт "Сменить среду выполнения" (Change runtime type).

В открывшемся окне в выпадающем списке "Аппаратный ускоритель" (Hardware accelerator) выберите желаемый вариант:

None (нет, только CPU)

GPU

TPU

Нажмите "Сохранить". Изменения вступят в силу после перезапуска среды выполнения.

Вывод: в ходе работы были исследованы базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.